RV College of Engineering, Bengaluru - 560059.

Go, change the world

Title: Full Stack E-Commerce Platform

Tanush Pandya | Vaidehi Gupta
1RV25CS189 | 1RZ25CS196

Introduction

ShopEasy is an Amazon- and Flipkart-inspired e-commerce platform where customers browse, filter, cart, and track orders, while sellers and admins manage listings and orders via a dashboard. Built as a three-tier architecture (vanilla HTML/CSS/JS frontend, Node.js/Express REST API, and MongoDB), all client data calls route through a single api.js file, enabling a seamless transition from mock data to the live backend.

Problem Definition

Full e-commerce platforms can be overwhelming to study or build. A functional store demands several synchronized layers:

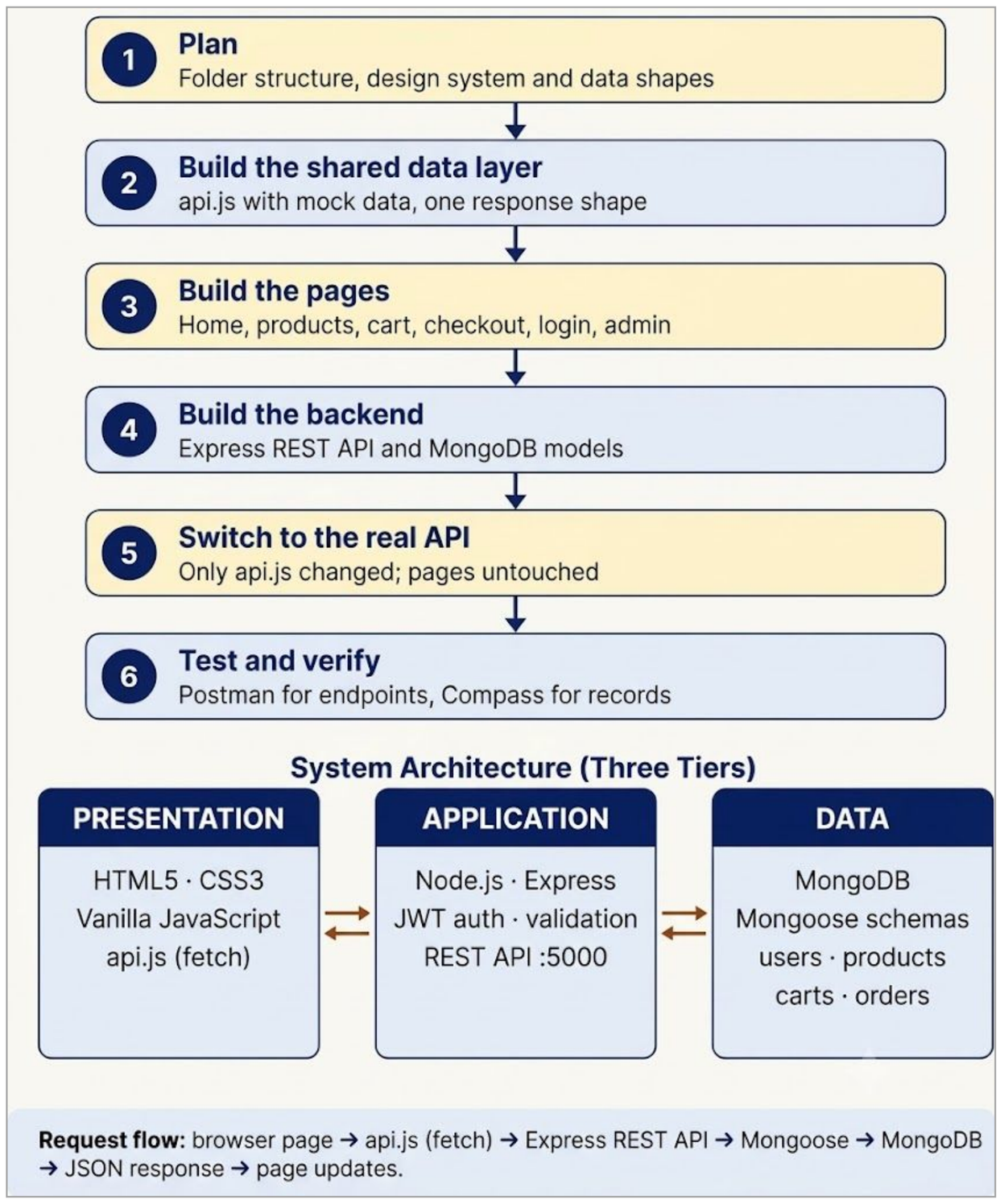
- **Responsive UI:** Adapts across mobile and desktop.
- **Role-Based Auth:** Secures logins and access levels.
- **Catalog:** Handles search, filters, sorting, and pagination.
- **Cart & Checkout:** Keeps stock and order totals accurate.
- **Database:** Persists users, products, and transactions.

This project implements every layer end-to-end to demystify how a full-stack store connects.

Objectives

1. Build a responsive, framework-free UI using vanilla HTML, CSS, and JS.
2. Design a clean REST API for seamless frontend integration.
3. Persist users, products, carts, and orders in MongoDB.
4. Implement secure authentication and role-based access control (RBAC).
5. Enforce server-side validation to prevent client-side price and stock tampering..
6. Follow a professional Git workflow with feature branches and clear commits.

Methodology



Tools used

Frontend	**HTML5**, **CSS3** (Flexbox, Grid), **vanilla JavaScript** (ES6+). Build semantic, responsive, framework-free pages to learn core DOM and events first-hand.
Backend	**Node.js**, **Express.js**. Single language across the stack; lightweight routing with middleware for auth, errors, and CORS.
Database	**MongoDB**, **Mongoose**. Document-based storage with natural mapping for schemas and data validation.
Security	**JWT**, **bcrypt**, **CORS**, **dotenv**. Stateless auth, hashed passwords, origin restrictions, and secure environment variables.
Workflow	**Postman, MongoDB Compass, Git/GitHub, Docker Compose, VS Code.** API testing, database UI, version control, and containerized dev environments.

Results and Discussions

The project delivered a full-stack e-commerce prototype balancing clean frontend architecture with reliable backend integrity. Across 9 pages, 20+ REST endpoints, and 4 MongoDB collections, all client requests route through a single api.js layer to provide smooth consumer and merchant experiences.

- **Decoupled Architecture:** 9 responsive pages (320px to desktop) driven by 20+ REST endpoints, 4 collections, and a unified api.js data layer.
- **Role-Based Workflows:** Distinct RBAC journeys for customers (search, cart, checkout, order tracking) and sellers (revenue metrics, catalog CRUD, order management).
- **Server-Side Integrity:** Server-calculated totals and atomic stock decrements with automated rollbacks to prevent client-side price or inventory tampering.
- **Dual Validation & Verification:** Synchronized client/server validation rules, tested and verified end-to-end via Postman and MongoDB Compass.
- **Security Controls:** Bcrypt password hashing, JWT authentication, XSS protection, and strict schema validation.

Conclusions

ShopEasy demonstrates that a robust, production-ready e-commerce platform can be built from scratch without relying on heavy frontend frameworks. By adhering to a modular three-tier architecture and enforcing strict server-side validation, the project successfully bridges the gap between learning core web fundamentals and delivering a secure, scalable full-stack application.

Outcome of the work

Product Development: a working, responsive and secure full-stack e-commerce prototype for customers and sellers, with source code version-controlled on GitHub using feature branches.

References

1. MDN Web Docs, HTML, CSS and JavaScript reference, developer.mozilla.org.
2. Express.js documentation, expressjs.com.
3. MongoDB Manual and Mongoose documentation, mongodb.com/docs, mongoosejs.com.
4. M. Jones, J. Bradley, N. Sakimura, JSON Web Token (JWT), RFC 7519, IETF, 2015.
5. N. Provos, D. Mazières, “A Future-Adaptable Password Scheme,” USENIX, 1999 (bcrypt).
6. OWASP Top 10 Web Application Security Risks, owasp.org.